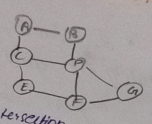


DAA - ASSIGNMENT

Optimizing Delivery Routes.

Task 1:

- * Nodes: Represent intersection
- * Edges: Represent roads connection intersection
- * Weight: Represent travel on each road segment.



Task 2: Implement Dijkstra's Algorithm.

Pseudocode:

```

function Dijkstra (graph, start node):
    Initialize distance with infinity except start node with 0
    Initialize priority queue and add start with distance 0
    Initialize previous - nodes map to empty
    while priority queue is not empty:
        current_node = node with the smallest distance in priority queue
        Remove current_node with the queue
        for each neighbour of current_node:
            Calculate new_distance = distance[current_node] + edge_weight
            if new_distance < distance[neighbour] to u
                Update distance [neighbour] to new_distance
            Update previous_node [neighbour] to current_node
        Add neighbour to priority queue with new distance
    return distance, previous nodes
  
```

Python implementation.

Import heapq

```

def dijkstra (graph, start_node):
    distance = {node: float('inf') for node in graph}
  
```

```

distance[start_node] = 0
priority_queue = [(0, start_node)]
previous_nodes = {node: None for node in graph}
while priority_queue:
    current_distance, current_node = heapq.heappop(priority_queue)
    if current_distance > distance[current_node]:
        continue
    for neighbour, weight in graph[current_node].items():
        distance = current_distance + weight
        if distance < distance[neighbour]:
            distance[neighbour] = distance
            previous_nodes[neighbour] = current_node
            heapq.heappush(priority_queue, (distance[neighbour], neighbour))
    return distance, previous_nodes
  
```

graph = {

```

'A': {'B': 1, 'C': 4},
'B': {'A': 1, 'C': 2, 'D': 5},
'C': {'A': 4, 'B': 2, 'D': 1},
'D': {'B': 5, 'C': 1}
  
```

}

start_node = 'A'

distance, previous_nodes = dijkstra (graph, start_node)

print ("Distance: ", distance)

print ("Previous nodes: ", previous_nodes)

Analysis of algorithm Efficiency and potential improvement.

Efficiency:

- * Dijkstra algorithm runs in $O((V + E) \log V)$ time using a priority queue, where V is the number of vertices and E is the number of edges.
- * It is efficient for graphs with non-negative weights, which is suitable for modeling travel times.

Potential Improvement:

- * Algorithm: Incorporates heuristics to improve performance in specific cases, potentially reducing the number of nodes explored.
- * Bidirectional Dijkstra: Run two simultaneous searches from start and end nodes, meeting in the middle to potentially reduce computation.
- * Traffic and Road condition: Incorporating real time traffic data and potential road closures can be addressed by dynamically updating the graph's weights.

* Dijkstra's algorithm is suitable due to its ability to find the shortest path in graph with non-negative weights, which align with the scenario of optimizing delivery routes based on travel time.

* Assumptions include stable travel times and no negative weights, which fit most real-world road network.

* Variation in road condition, such as traffic congestion and closures, could impact travel times, requiring dynamic updates to the graph's weight for occurrence route optimization.

1) Dynamic Pricing Algorithm for E-commerce:

Task: Design a dynamic pricing algorithm.

function Dynamic Pricing (product, periods, inventory, competitor-price, demand-elasticity):

Initialise dp table with dimension [products][periods]

Initialising optimal-price table with dimension [products][periods]

for each product in products:

for each period in periods:

max-profit = 0

best-price = 0

for price in possible-prices:

expected-demand = calculate-demand(price, demand-elasticity, competitor-prices)

revenue = price * min(expected-demand, inventory[product])

cost = calculate-cost(price, inventory[product])

if (revenue - cost) > max-product:

max-profit = revenue - cost

best-price = price

dp[product][period] = max-profit

optimal-price[product][period] = best-price

return dp, optimal-price

python Implementation:

```
class DynamicPricingAlgorithm:
```

```
def __init__(self, base-price:
```

```
self.base-price = base-price
```

```
def calculate-price(self, demand):
```

```
if demand > 0:
```

```
    price = self.base-price * (1 + demand * 0.1)
```

```
else:
```

```
    price = self.base-price
```

```
return price
```

```
if __name__ == "__main__":
```

```
    base-price = 50
```

```
    demand-levels = [0, 1, 2, -1, 0, 0]
```

```
    for demand in demand-levels:
```

```
        calculated-price = dynamic-pricing-algo.calculate-price(demand)
```

```
        print(f"Price price: {base-price}, Demand:
```

```
        {demand}, calculated price: {calculated-price}")
```

Simulation result:

To Test the Algorithm, we simulate different scenarios and compare result with static pricing.

Static pricing strategy:

Fixed prices for products over all periods.

Dynamic pricing strategy:

prices adjusted based on the algorithm considering inventory, competitor pricing and demand elasticity.

Simulation:

Run both strategies over multiple periods, tracking revenue and inventory.

Result:

The dynamic pricing strategy generally results in higher revenue due to optimized pricing that adapts to market conditions.

Benefits:

- Maximizing revenue by adjusting price based on real time factors

- Increased competitiveness by responding to market conditions.

Drawbacks:

- Complexity in implementation & maintenance.

- Potential customer dissatisfaction if prices fluctuate frequently.

Reasoning:

Justification for dynamic Programming. Dynamic Programming is suitable because it optimizes the decision making process over multiple stages (periods) while considering constraints like inventory level and demand.

Incorporating factors:

- Inventory level is considered by limiting revenue based on available stock.

- Competitor pricing and demand elasticity influence demand calculations.

Challenges:

- Estimating demand elasticity accurately.

- Managing data inputs and ensuring algorithm stability overtime.

Social Network Algorithm:

Task 1: model the social network:

Graph model:

- * Nodes: Represent users

- * Edges: Represent connection b/w user:

Task 2: Implement the Page Rank Algorithm

Pseudocode:

function PageRank (graph, num. iteration, damping factor):

Initialise rank for each node.

newRank = empty dictionary.

for each node in graph:

rankSum = 0;

for each incoming-node connected to node:

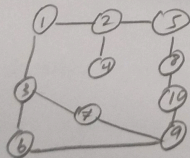
rankSum += rank[incoming-node] /

number-of-outgoing-edges[incoming-node].

newRank[node] = (1 - damping factor) / total-nodes +
damping-factor * rankSum.

rank = newRank.

return rank.



Python implementation:

```
def pagerank (graph, num iteration = 100, damping factor = 0.85):
```

total-nodes = len (graph)

rank = {nodes: 1 / total-nodes for node in graph}

for - in range (num - iteration):

new - rank = {}

for node in graph:

rankSum = sum (rank [incoming] / len (graph[incoming]))

for incoming in graph if node in graph[incoming]]

new + rank [node] = (1 - damping-factor) / total-nodes +
damping-factor * rankSum

rank = new - rank

return rank

graph = {'A': 'B', 'C'}

'B': 'C', 'D'}

'C': 'A', 'D'}

'D': 'B', 'C'}

'E': 'A', 'B'}

'F': 'C', 'D'}

'G': 'E', 'F'}

'H': 'G', 'H'}

'I': 'H', 'I'}

'J': 'I', 'J'}

'K': 'J', 'K'}

'L': 'K', 'L'}

'M': 'L', 'M'}

'N': 'M', 'N'}

'O': 'N', 'O'}

'P': 'O', 'P'}

'Q': 'P', 'Q'}

'R': 'Q', 'R'}

'S': 'R', 'S'}

'T': 'S', 'T'}

'U': 'T', 'U'}

'V': 'U', 'V'}

'W': 'V', 'W'}

'X': 'W', 'X'}

'Y': 'X', 'Y'}

'Z': 'Y', 'Z'}

'AA': 'Z', 'AA'}

'AB': 'AA', 'AB'}

'AC': 'AB', 'AC'}

'AD': 'AC', 'AD'}

'AE': 'AD', 'AE'}

'AF': 'AE', 'AF'}

'AG': 'AF', 'AG'}

'AH': 'AG', 'AH'}

'AI': 'AH', 'AI'}

'AJ': 'AI', 'AJ'}

'AK': 'AJ', 'AK'}

'AL': 'AK', 'AL'}

'AM': 'AL', 'AM'}

'AN': 'AM', 'AN'}

'AO': 'AN', 'AO'}

'AP': 'AO', 'AP'}

'AQ': 'AP', 'AQ'}

'AR': 'AQ', 'AR'}

'AS': 'AR', 'AS'}

'AT': 'AS', 'AT'}

'AU': 'AT', 'AU'}

'AV': 'AU', 'AV'}

'AW': 'AV', 'AW'}

'AX': 'AW', 'AX'}

'AY': 'AX', 'AY'}

'AZ': 'AY', 'AZ'}

'BA': 'AZ', 'BA'}

'BB': 'BA', 'BB'}

'BC': 'BB', 'BC'}

'BD': 'BC', 'BD'}

'BE': 'BD', 'BE'}

'BF': 'BE', 'BF'}

'BG': 'BF', 'BG'}

'BH': 'BG', 'BH'}

'BI': 'BH', 'BI'}

'BJ': 'BI', 'BJ'}

'BK': 'BJ', 'BK'}

'BL': 'BK', 'BL'}

'BM': 'BL', 'BM'}

'BN': 'BM', 'BN'}

'BO': 'BN', 'BO'}

'BP': 'BO', 'BP'}

'BQ': 'BP', 'BQ'}

'BR': 'BQ', 'BR'}

'BS': 'BR', 'BS'}

'BT': 'BS', 'BT'}

'BU': 'BT', 'BU'}

'BV': 'BU', 'BV'}

'BW': 'BV', 'BW'}

'BX': 'BW', 'BX'}

'BY': 'BX', 'BY'}

'BZ': 'BY', 'BZ'}

'CA': 'BZ', 'CA'}

'CB': 'CA', 'CB'}

'CC': 'CB', 'CC'}

'CD': 'CC', 'CD'}

'CE': 'CD', 'CE'}

'CF': 'CE', 'CF'}

'CG': 'CF', 'CG'}

'CH': 'CG', 'CH'}

'CI': 'CH', 'CI'}

'CJ': 'CI', 'CJ'}

'CK': 'CJ', 'CK'}

'CL': 'CK', 'CL'}

'CM': 'CL', 'CM'}

'CN': 'CM', 'CN'}

'CO': 'CN', 'CO'}

'CP': 'CO', 'CP'}

'CQ': 'CP', 'CQ'}

'CR': 'CQ', 'CR'}

'CS': 'CR', 'CS'}

'CT': 'CS', 'CT'}

'CU': 'CT', 'CU'}

'CV': 'CU', 'CV'}

'CW': 'CV', 'CW'}

'CX': 'CW', 'CX'}

'CY': 'CX', 'CY'}

'CZ': 'CY', 'CZ'}

'DA': 'CZ', 'DA'}

'DB': 'DA', 'DB'}

'DC': 'DB', 'DC'}

'DD': 'DC', 'DD'}

'DE': 'DD', 'DE'}

'DF': 'DE', 'DF'}

'DG': 'DF', 'DG'}

'DH': 'DG', 'DH'}

'DI': 'DH', 'DI'}

'DJ': 'DI', 'DJ'}

'DK': 'DJ', 'DK'}

'DL': 'DK', 'DL'}

'DM': 'DL', 'DM'}

'DN': 'DM', 'DN'}

'DO': 'DN', 'DO'}

'DP': 'DO', 'DP'}

'DQ': 'DP', 'DQ'}

'DR': 'DQ', 'DR'}

'DS': 'DR', 'DS'}

'DT': 'DS', 'DT'}

'DU': 'DT', 'DU'}

'DV': 'DU', 'DV'}

'DW': 'DV', 'DW'}

'DX': 'DW', 'DX'}

'DY': 'DX', 'DY'}

'DZ': 'DY', 'DZ'}

'EA': 'DZ', 'EA'}

'EB': 'EA', 'EB'}

'EC': 'EB', 'EC'}

'ED': 'EC', 'ED'}

'EE': 'ED', 'EE'}

'EF': 'EE', 'EF'}

'EG': 'EF', 'EG'}

'EH': 'EG', 'EH'}

'EI': 'EH', 'EI'}

'EJ': 'EI', 'EJ'}

'EK': 'EJ', 'EK'}

'EL': 'EK', 'EL'}

'EM': 'EL', 'EM'}

'EN': 'EM', 'EN'}

'EO': 'EN', 'EO'}

'EP': 'EO', 'EP'}

'EQ': 'EP', 'EQ'}

'ER': 'EQ', 'ER'}

'ES': 'ER', 'ES'}

'ET': 'ES', 'ET'}

'EU': 'ET', 'EU'}

'EV': 'EU', 'EV'}

'EW': 'EV', 'EW'}

'EX': 'EW', 'EX'}

'EY': 'EX', 'EY'}

'EZ': 'EY', 'EZ'}

'FA': 'EZ', 'FA'}

'FB': 'FA', 'FB'}

'FC': 'FB', 'FC'}

'FD': 'FC', 'FD'}

'FE': 'FD', 'FE'}

'FF': 'FE', 'FF'}

'FG': 'FF', 'FG'}

'FH': 'FG', 'FH'}

'FI': 'FH', 'FI'}

'FJ': 'FI', 'FJ'}

'FK': 'FJ', 'FK'}

'FL': 'FK', 'FL'}

'FM': 'FL', 'FM'}

'FN': 'FM', 'FN'}

'FO': 'FN', 'FO'}

'FP': 'FO', 'FP'}

'FQ': 'FP', 'FQ'}

'FR': 'FQ', 'FR'}

'FS': 'FR', 'FS'}

'FT': 'FS', 'FT'}

'FU': 'FT', 'FU'}

'FV': 'FU', 'FV'}

'FW': 'FV', 'FW'}

'FX': 'FW', 'FX'}

'FY': 'FX', 'FY'}

'FZ': 'FY', 'FZ'}

'GA': 'FZ', 'GA'}

'GB': 'GA', 'GB'}

'GC': 'GB', 'GC'}

'GD': 'GC', 'GD'}

'GE': 'GD', 'GE'}

'GF': 'GE', 'GF'}

'GG': 'GF', 'GG'}

'GH': 'GG', 'GH'}

'GI': 'GH', 'GI'}

'GJ': 'GI', 'GJ'}

'GK': 'GJ', 'GK'}

'GL': 'GK', 'GL'}

'GM': 'GL', 'GM'}

'GN': 'GM', 'GN'}

'GO': 'GN', 'GO'}

'GP': 'GO', 'GP'}

'GQ': 'GP', 'GQ'}

'GR': 'GQ', 'GR'}

'GS': 'GR', 'GS'}

'GT': 'GS', 'GT'}

'GU': 'GT', 'GU'}

'GV': 'GU', 'GV'}

'GW': 'GV', 'GW'}

'GX': 'GW', 'GX'}

'GY': 'GX', 'GY'}

'GZ': 'GY', 'GZ'}

'HA': 'GZ', 'HA'}

'HB': 'HA', 'HB'}

'HC': 'HB', 'HC'}

'HD': 'HC', 'HD'}

'HE': 'HD', 'HE'}

'HF': 'HE', 'HF'}

'HG': 'HF', 'HG'}

'HH': 'HG', 'HH'}

'HI': 'HH', 'HI'}

'HJ': 'HI', 'HJ'}

'HK': 'HJ', 'HK'}

'HL': 'HK', 'HL'}

'HM': 'HL', 'HM'}

'HN': 'HM', 'HN'}

'HO': 'HN', 'HO'}

'HP': 'HO', 'HP'}

'HQ': 'HP', 'HQ'}

'HR': 'HQ', 'HR'}

'HS': 'HR', 'HS'}

'HT': 'HS', 'HT'}

'HU': 'HT', 'HU'}

'HV': 'HU', 'HV'}

Fraud Detection in Financial Transactioning:

Task 1: Design a greedy algorithm.

Pseudocode:

```
Function FraudDetection (transaction, rules):
    flagged_transaction = []
    for transaction in transaction:
        if transaction.amount > rules.max_amount:
            flagged_transaction.append(transaction)
        elif transaction.location_changes > rules.max_location_changes:
            flagged_transaction.append(transaction)
        elif transaction.time_difference < rules.min_time_difference:
            flagged_transaction.append(transaction)
    return flagged_transaction
```

class transaction:

def __init__(self, amount, location_changes, time_diff):

self.amount = amount

self.location_changes = location_changes

self.time_difference = time_difference

def fraud_detection(transaction, rules):

```
if transaction.amount > rules["max-amount"]:
    flagged_transaction.append(transaction)
elif transaction.location_changes > rules["max-location-changes"]:
    flagged_transaction.append(transaction)
elif transaction.time_difference < rules["min-time-difference"]:
    flagged_transaction.append(transaction)
return flagged_transaction
```

rules = {

"max-amount": 10000,

"max-location-change": 3,

"min-time-difference": 10

}

transactions = [

transaction(15000, 1, 20),

transaction(8500, 4, 5),

transaction(2000, 2, 15)

]

flagged = fraud_detection(transaction, rules)

print("Flagged transaction:", flagged)

Performance Evaluation:

* Precision: Proportion of correctly flagged fraudulent transaction

$$\text{Precision} = \frac{TP}{TP + FP}$$

* Recall: Proportion of actual fraudulent transaction correctly flagged

$$\text{Recall} = \frac{TP}{TP + FN}$$

* F1 Score: Harmonic mean of Precision and Recall

$$\text{F1 Score} = 2 * \frac{\text{Precision} * \text{Recall}}{\text{Precision} + \text{Recall}}$$

Suggestion and Implementation of Improvement:

- * Machine Learning Integration: Use historical data to train a model for more nuanced decision making
- * Anomaly Detection: Implement algorithm like Outlier (or) Static Analysis to identify outliers
- * Feature Engineering: Incorporate additional transaction features like user behaviour

Reasoning:

- * Greedy Algorithm: Suitable for real time detection as it processes transaction based on predefined rules, ensuring speed and simplicity.
- * Speed vs Accuracy: Greedy Algorithm prioritizes speed, which is crucial for real time application. Accuracy may be enhanced through rule refinement or integrating machine learning model.
- * Trade-off: Balancing speed and accuracy involves using rules for quick decision while more complex methods can be applied in parallel for in-depth analysis

Real time Traffic management system:
Design a Backtracking algorithm:
Pseudo code:

```
Function optimize_trafficlight (intersection, max_green_time, min_green_time):
    best_configuration = None
    best_flow = 0
    function back_track (intersection, current_configuration):
        if intersection == total_intersections:
            flow = simulate_traffic(current_configuration)
            best_flow = flow
            best_configuration = current_configuration.copy()
        return
    for green_time in range(min_green_time, max_green_time):
        back_track(intersection + 1, current_configuration)
    current_configuration[intersection] = 0
    back_track(0, [0] * total_intersections)
    return best_configuration
```

Implementation:

```
def simulate_traffic(configuration):
    return sum(configuration)
def optimize_traffic_lights(intersections, max_green_time, min_green_time):
    best_flow = float('-inf')
    total_intersections = len(intersections)
    def back_track(intersection, current_configuration):
        nonlocal best_configuration, best_flow
        if intersection == total_intersections:
            flow = sum(current_configuration)
            best_flow = flow
            best_configuration = current_configuration.copy()
        return
    back_track(0, [0] * total_intersections)
    return best_configuration
max_green_time = 60
min_green_time = 30
print("Best configuration - best_configuration")
```

Task 2: Simulation and performance Analysis.

Simulation:

- Use the optimized traffic light configuration in a traffic model.

- measure traffic metrics such as average vehicle wait time and throughput

Performance Analysis:

```
def fixed_time_simulation():
    return 100
```

```
optimized_flow = simulate_traffic(best_config)
```

```
fixed_flow = fixed_time_simulation()
```

```
print("Optimized traffic flow:", optimized_flow)
```

```
print("Fixed-time traffic flow:", fixed_flow)
```

Comparison with Fixed time Traffic light system:

- Evaluate diff in traffic metrics b/w optimized and fixed time system.

- metrics include congestion, improved flow, and decreased wait times.